

Sistema de Predicción de Aprobación de Tarjetas de Crédito usando Aprendizaje Automático

Valentina C. Zapata,¹ Luis G. Sánchez², y Jose D. Gómez Muñetón.³

¹valentina.candenaz@udea.edu.co ²guillermo.sanchez1@udea.edu.co

³jose.gomez14@udea.edu.co

Palabras Clave—Modelos predictivos, Árboles de decisión, Regresión logística, Riesgo crediticio, Aprendizaje supervisado.

I. INTRODUCCIÓN

EN el presente proyecto se propone desarrollar un modelo de aprendizaje automático (*Machine Learning*) capaz de predecir si un solicitante será un “buen” o “mal” cliente. Este enfoque busca automatizar y optimizar el proceso de aprobación de tarjetas de crédito, permitiendo al banco reducir riesgos y mejorar la eficiencia del proceso de evaluación. La solución basada en *Machine Learning* permitirá no solo realizar predicciones, sino también adaptarse mejor a nuevas condiciones económicas o cambios en los patrones de comportamiento de los usuarios.

II. DESCRIPCIÓN DEL PROBLEMA

La aprobación de tarjetas de crédito es un proceso crítico para las entidades financieras, ya que implica una evaluación cuidadosa del riesgo crediticio asociado a cada solicitante. Con el objetivo de mitigar el riesgo de impago, los bancos utilizan modelos predictivos basados en información personal, laboral y financiera de los solicitantes para decidir si aprueban o no una solicitud. Tradicionalmente, esta evaluación se realiza a través de puntuaciones crediticias construidas a partir de modelos estadísticos como la regresión logística.

El desarrollo de una solución basada en *Machine Learning* ofrece ventajas significativas sobre los métodos tradicionales, ya que estos modelos pueden captar patrones complejos y no lineales en los datos que los enfoques estadísticos convencionales podrían pasar por alto. Además, los algoritmos de *Machine Learning* tienen la capacidad de aprender y mejorar con el tiempo a medida que se dispone de nuevos datos, lo que permite una adaptación dinámica a cambios en el comportamiento de los solicitantes o en las características del perfil de los clientes objetivo.

El conjunto de datos utilizado proviene del portal Kaggle [1] y se compone de dos archivos principales: *application_record.csv* y *credit_record.csv*. El primero contiene información detallada sobre los solicitantes, incluyendo datos demográficos, laborales y financieros, mientras que el segundo recoge el comportamiento crediticio a lo largo del tiempo.

La información demográfica proporciona datos clave sobre el perfil del solicitante y permite comprender aspectos sociales y personales que pueden influir en el comportamiento crediticio a lo largo del tiempo. Por otra parte, se incluyen también

variables de carácter económico y laboral, como el nivel de ingresos, el tipo de empleo, el nivel educativo y la ocupación, que ayudan a estimar la estabilidad financiera del solicitante y su capacidad para generar ingresos de manera sostenida.

Asimismo, se consideran datos sobre propiedades y activos, tales como la posesión de vivienda, automóvil y número de miembros dependientes, los cuales permiten evaluar el patrimonio y las responsabilidades económicas del solicitante. Finalmente, el historial crediticio, registrado en *credit_record.csv*, proporciona una visión temporal del comportamiento de pago, incluyendo retrasos y cumplimiento de pagos.

En conjunto, estas variables reflejan la solvencia económica del solicitante, lo cual es fundamental para evaluar su capacidad de pago y, por ende, el riesgo crediticio asociado a la aprobación de una tarjeta de crédito.

En total, se cuenta con 438,557 registros de información de clientes, descritos mediante 17 variables, y 1,048,575 registros en el historial de créditos. Las variables son de tipo categórico, numérico y binario. Entre las variables categóricas se incluyen el género, tipo de ingreso, nivel educativo, estado civil, tipo de vivienda y ocupación; las variables numéricas abarcan el número de hijos, ingresos totales, edad y años trabajados; mientras que las variables binarias indican características como si el solicitante posee coche o vivienda.

Durante el análisis exploratorio se identificaron valores faltantes en la variable *OCCUPATION_TYPE* (ocupación). Por ello, se propone imputar dicha variable con la categoría “Desconocido”. Adicionalmente, se sugiere contrastar estos datos con la variable de días de desempleo, con el fin de imputar los valores faltantes como “Desempleado” en aquellos casos donde se registre un periodo de desempleo.

En cuanto a la codificación de variables, se aplicará codificación ordinal para aquellas que presenten una jerarquía inherente, como el nivel educativo, y codificación one-hot para el resto de las variables categóricas. Asimismo, se llevará a cabo escalamiento o normalización en las variables numéricas que lo requieran, como *AMT_INCOME_TOTAL* (ingresos anuales), con el objetivo de mejorar el rendimiento de los modelos.

Un reto importante de este dataset es la ausencia de una etiqueta explícita que indique si la solicitud fue aprobada o si el cliente fue bueno o malo. Por lo tanto, se debe construir una variable objetivo a partir de la información contenida en *credit_record.csv*. Una alternativa es etiquetar como “buen cliente” a aquel que no presenta atrasos significativos (por ejemplo, no más de dos meses de mora en los pagos), y como “mal cliente” a aquellos con historial negativo recurrente.

El problema a resolver es de clasificación binaria, y se evaluarán diferentes modelos del aprendizaje automático. En primer lugar, se considerará la regresión logística debido a su facilidad de interpretación y su uso frecuente en el ámbito financiero. No obstante, este modelo puede no ser suficiente si las relaciones entre variables no son lineales. En ese caso, se recurrirá a modelos de árboles de decisión, como el *Random Forest*, que son más robustos frente a datos mixtos y no requieren un extenso preprocesamiento. Finalmente, se tendrá en cuenta el uso de modelos de gradiente como *XGBoost*, ampliamente reconocidos por su precisión en competencias y aplicaciones reales, aunque más complejos en su ajuste y explicación. La comparación entre estos modelos permitirá seleccionar aquel que logre el mejor balance entre precisión, interpretabilidad y eficiencia.

Es importante tener en cuenta el desbalance de clases que se puede presentar al construir la variable objetivo. Este fenómeno puede llevar a que los modelos predican más frecuentemente la clase mayoritaria, afectando su desempeño real. Para mitigarlo, se planea aplicar técnicas como la reponderación de clases, el sobremuestreo de la clase minoritaria mediante métodos como SMOTE, o la utilización de métricas como el *F1-score* o el AUC-ROC, más adecuadas en escenarios desbalanceados.

III. ESTADO DEL ARTE

Para abordar el problema de predicción de aprobación de tarjetas de crédito, se analizó el artículo titulado “*Predicting Credit Card Approval Using Machine Learning Techniques*” de Ehsan Lotfi (2024) [2], publicado en la revista *International Journal of Applied Data Science in Engineering and Health*. En este artículo se aborda el problema de aprobación de tarjetas de crédito empleando el mismo dataset usado para este estudio.

La variable objetivo en este estudio, que busca predecir la aprobación de tarjetas de crédito, es de naturaleza binaria (aprobado o rechazado) y presenta un desbalance significativo en sus clases. Su construcción se basa fundamentalmente en el historial crediticio del solicitante, extraído del conjunto de datos *credit_record.csv*, donde el campo *STATUS* es crucial al proporcionar información detallada sobre el estado del crédito y el comportamiento de pago, permitiendo así definir el resultado de la aprobación, sin embargo, no se confirma cuál es el umbral final elegido la construcción de la variable.

En cuanto al paradigma de aprendizaje, se utiliza el enfoque de aprendizaje supervisado, ya que el modelo se entrena con ejemplos etiquetados que indican si la solicitud de tarjeta fue aprobada o no. Para resolver el problema, los autores comparan cuatro algoritmos de aprendizaje automático: Regresión Logística, Árbol de Decisión, *Random Forest* y *XGBoost* (*Extreme Gradient Boosting*). Cada uno de estos métodos tiene distintas características en cuanto a complejidad, capacidad de generalización e interpretación.

La metodología de validación consiste en una partición del dataset en 80 % para entrenamiento y 20 % para prueba. Además, se utilizaron técnicas de validación cruzada y *Random Search* para el ajuste de hiperparámetros del modelo *XGBoost*. Esto permitió optimizar variables como tasa

de aprendizaje, profundidad máxima del árbol, número de estimadores y penalizaciones L1 y L2.

Respecto a las métricas de evaluación, se emplearon varias para medir el rendimiento del sistema: *accuracy* (exactitud), *precision* (precisión), *recall* (sensibilidad), *F1-score* (media armónica entre precisión y *recall*), y la matriz de confusión. Estas métricas permiten analizar no solo cuántas predicciones fueron correctas, sino también la calidad de las predicciones positivas, lo cual es clave en contextos donde hay desbalance de clases.

Los resultados mostraron que el modelo *XGBoost* fue el más efectivo, alcanzando un 99.04 % de exactitud, 85 % de *recall* y 78 % de precisión sobre el conjunto de prueba. Estos valores evidencian su capacidad para identificar correctamente a los solicitantes aprobados y minimizar errores tipo I y II. El artículo concluye que los métodos de ensamble avanzados como *XGBoost* y *Random Forest* superan claramente a modelos más simples como la regresión logística, particularmente cuando se trata de capturar relaciones no lineales y patrones complejos en grandes volúmenes de datos.

Después de esto se realizó el análisis de un segundo artículo de Chang et al. (2024), titulado “*Credit Risk Prediction Using Machine Learning and Deep Learning: A Study on Credit Card Customers*” [3], el cual aborda el problema utilizando el mismo dataset.

El criterio de “buen” y “mal” cliente, definido por la mora de la deuda de 60 días o más, es la base fundamental para la construcción de la variable objetivo. Dicha clasificación establece directamente si un solicitante es aprobado o rechazado, siendo un indicador cuantificable y determinante para la definición de la variable.

El trabajo utiliza aprendizaje supervisado, ya que se entrena un modelo para predecir si un cliente será “bueno” o “malo” (es decir, si tendrá o no problemas de pago) a partir de datos históricos etiquetados. En este caso se probaron seis algoritmos de clasificación: regresión logística, *Random Forest*, redes neuronales (ANN), *AdaBoost*, *LightGBM* y *XGBoost*.

El conjunto de datos se dividió en un 70 % para entrenamiento y 30 % para prueba. Se utilizó la técnica SMOTE (*Synthetic Minority Oversampling Technique*) para abordar el fuerte desbalance de clases (solo el 1.3 % de los clientes eran “malos”). Aunque no se aplicó validación cruzada (K-fold), se recomienda como trabajo futuro.

Las métricas utilizadas fueron *accuracy*, *precision*, *recall*, *F1-score*, ROC-AUC y MCC (*Matthews Correlation Coefficient*). Esta última es útil para problemas con clases desbalanceadas, ya que considera todos los valores de la matriz de confusión. Un MCC de 1 indica una clasificación perfecta.

Como resultado se obtuvo que el modelo *XGBoost* fue el más preciso, con un *accuracy* del 99.3 %, *precision* y *recall* de 0.993, *F1-score* de 0.993, AUC de 0.997 y MCC de 0.986. *LightGBM* obtuvo resultados similares. Las redes neuronales, en cambio, tuvieron un rendimiento inferior debido al desbalance de clases.

Como tercera referencia se analizó el artículo de Caggiano y Giardini (2024) [4], titulado “*Machine Learning Analysis of Credit Card Approval*”, el cual aborda la problemática de la aprobación de tarjetas de crédito desde una perspectiva bancaria, donde identificar clientes “buenos” o “malos” representa una decisión crítica.

El estudio emplea técnicas de *Machine Learning* para predecir qué usuarios podrían representar una mala inversión para el banco, basándose en información socioeconómica y en el comportamiento histórico de pago. Los autores resaltan que, ante el creciente uso de tarjetas de crédito, las entidades financieras necesitan modelos automatizados más precisos que superen las limitaciones del simple *score* crediticio tradicional.

Para construir la variable objetivo, el estudio utilizó dos conjuntos de datos provenientes de *Kaggle*: uno con información demográfica y económica de los solicitantes, y otro con el historial mensual de pagos de sus tarjetas. Se etiquetó cada mes como “malo” si hubo mora superior a 30 días, y luego se calculó el porcentaje de meses en mora por tarjeta. Si ese porcentaje superaba el 10 %, se consideraba una tarjeta “mala”. Luego, se agrupó la información por cliente y se definió como “cliente malo” a quien tuviera más del 15 % de sus tarjetas en estado “malo”. Esta variable binaria (“bueno” o “malo”) fue utilizada como target en los modelos de clasificación, aplicando algoritmos como regresión logística, *Random Forest*, *Naive Bayes*, MLP y *Naive Bayes Tree*, con técnicas como *SMOTE* y aprendizaje sensible al costo para mitigar el fuerte desbalance de clases.

Por último, se examinó el artículo de Wang (2023), titulado “*Credit Card Default Prediction with Data Modeling*” [5], el cual aborda la problemática del riesgo crediticio mediante la predicción de incumplimientos en el pago de tarjetas de crédito. Utilizando un conjunto de datos de *Kaggle*, compuesto por información demográfica, económica y comportamental de los solicitantes, el estudio propone la implementación de modelos de aprendizaje automático para anticipar qué clientes podrían representar una amenaza financiera para las entidades emisoras.

Se evaluaron cuatro algoritmos: regresión logística, *KNN*, *Random Forest* y *XGBoost*, con métricas como *AUC*, *F1-score*, precisión y *recall*. El objetivo principal fue minimizar las pérdidas bancarias derivadas de la aprobación errónea de clientes con alto riesgo de incumplimiento, estableciendo que *Random Forest* y *XGBoost* fueron los más eficaces, alcanzando *AUC* de 0.771 y 0.753 respectivamente. La variable objetivo fue construida a partir del historial de pagos mensuales de cada cliente, clasificado originalmente en ocho niveles que indicaban el estado de cumplimiento de la deuda.

Se redefinió como una variable binaria: los valores “0”, “C” y “X” (pagado a tiempo o sin préstamo activo) se agruparon como clase negativa (clientes buenos), mientras que cualquier estado de mora superior a 30 días fue asignado como clase positiva (clientes malos). Esta simplificación permitió evaluar el desempeño de los modelos en la detección de clientes con mayor probabilidad de incumplimiento, enfrentando el reto de un conjunto de datos desbalanceado donde la mayoría de usuarios cumplía con sus pagos. Para mitigar este problema, los autores exploraron ajustes en el umbral de clasificación y

validación cruzada para optimizar el rendimiento predictivo de los modelos.

IV. ENTRENAMIENTO Y EVALUACIÓN DE LOS MODELOS

A. Configuración Experimental

1) *Estrategia de Validación*: Para evaluar el rendimiento de los modelos de forma robusta, se utilizará **validación cruzada estratificada**, lo que permite mantener la proporción original de clases (clientes buenos y malos) en cada subconjunto de entrenamiento y validación. Esta técnica es especialmente útil en escenarios con clases desbalanceadas, asegurando que cada *fold* tenga una representación adecuada de ambas clases.

2) *Estrategia de Sobremuestreo*: El conjunto de datos presenta un desbalance considerable entre las clases objetivo. Para abordar esta problemática, se aplicará la técnica de **SMOTE (*Synthetic Minority Over-sampling Technique*)**. SMOTE genera nuevas instancias sintéticas para la clase minoritaria, mejorando así la capacidad del modelo para aprender patrones relevantes de dicha clase sin recurrir al simple duplicado de ejemplos existentes.

3) *Métricas de Evaluación*: Dada la naturaleza del problema de evaluar si una clasificación fue correcta o no, tener dos clases y sumado el desbalance de clases, se utilizarán las siguientes métricas para evaluar los modelos:

- **Accuracy**: Proporción de predicciones correctas sobre el total de predicciones realizadas. Aunque es una métrica común, puede ser engañosa en conjuntos de datos desbalanceados.
- **Precisión (*Precision*)**: Fracción de verdaderos positivos sobre el total de predicciones positivas.
- **Recall**: Fracción de verdaderos positivos sobre el total de positivos reales.
- **F1-Score**: Media armónica entre precisión y recall, útil para evaluar el equilibrio entre ambas.
- **AUC-ROC**: Área bajo la curva ROC, que mide la capacidad del modelo para distinguir entre clases.

4) *Hiperparámetros Evaluados*: Para optimizar el rendimiento de los modelos, se realiza una búsqueda en el espacio de hiperparámetros mediante la evaluación de múltiples combinaciones potenciales. Esta exploración permite identificar la configuración que ofrece los mejores resultados en términos de las métricas seleccionadas. La Tabla I muestra el rango de valores considerados para cada hiperparámetro según el tipo de modelo.

Para la **regresión logística**, se evaluaron combinaciones de penalizaciones λ_1 y λ_2 , utilizando los solvers `liblinear` y `saga`, junto con diferentes valores del parámetro C (0.01, 0.1, 1 y 10), el cual regula la fuerza de la regularización y contribuye a prevenir el sobreajuste. También se exploraron dos valores para `max_iter` (100 y 200), con el fin de asegurar la convergencia del modelo. Este procedimiento

Tabla I
CONJUNTO DE HIPERPARÁMETROS POR MODELO

Modelo	Hiperparámetros
Regresión Logística	max_iter: [100, 200] solver: [liblinear, saga] C: [0.01, 0.1, 1, 10] penalty: [l1, l2]
K-Nearest Neighbors	n_neighbors: [3, 5, 7, 9] weights: [uniform, distance] metric: [euclidean, manhattan]
XGBoost	n_estimators: [50, 100, 200] max_depth: [3, 6, 9] learning_rate: [0.01, 0.1, 0.2]
MLPClassifier	hidden_layer_sizes: [(50), (100), (50, 50)] activation: [relu, tanh] solver: [adam, sgd] alpha: [0.0001, 0.001] learning_rate_init: [0.001, 0.01] max_iter: [200]
SVC (SVM)	C: [0.1, 1, 10] kernel: [linear, rbf] gamma: [scale, auto]

fue implementado utilizando GridSearchCV con validación cruzada estratificada.

En el caso del modelo **K-Nearest Neighbors (KNN)**, se ajustaron parámetros como el número de vecinos (`n_neighbors`), el tipo de métrica de distancia (euclidean o manhattan) y el esquema de ponderación (uniform o distance). Estos factores inciden directamente en la sensibilidad del modelo a la vecindad local y la topología del espacio de características.

Para el **clasificador MLP (perceptrón multicapa)**, se probaron distintas configuraciones de capas ocultas (`hidden_layer_sizes`), funciones de activación (relu y tanh), optimizadores (adam y sgd), tasas de aprendizaje inicial (`learning_rate_init`) y valores del coeficiente de regularización alpha. También se fijó `max_iter` en 200 para asegurar la finalización del entrenamiento.

El modelo **XGBoost**, basado en el ensamble de árboles, fue ajustado mediante los hiperparámetros `n_estimators`, `max_depth` y `learning_rate`, que controlan respectivamente el número de árboles, la profundidad máxima de cada árbol y la tasa de aprendizaje en cada iteración del boosting.

Por último, para el modelo **SVC (máquinas de vectores de soporte)**, se exploraron diferentes combinaciones del parámetro C, los kernels linear y rbf, y el parámetro gamma, que afecta la forma del margen de decisión en el espacio transformado.

Esta estrategia de búsqueda sistemática fue clave para mejorar el desempeño de cada modelo y asegurar que las comparaciones entre ellos se realizaran bajo condiciones técnicas equivalentes y justas.

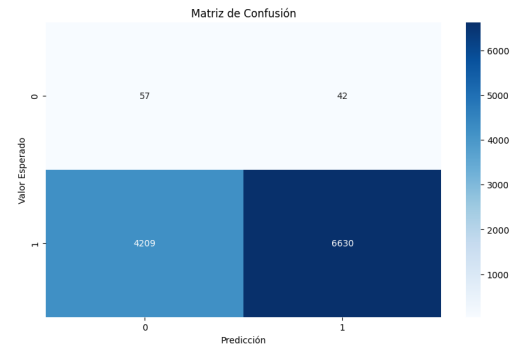


Figura 1. Este modelo acierta en 6,630 clientes buenos y 57 malos, pero presenta 4,209 falsos negativos. Aunque tiene un desempeño aceptable, su sensibilidad ante la clase minoritaria es limitada.

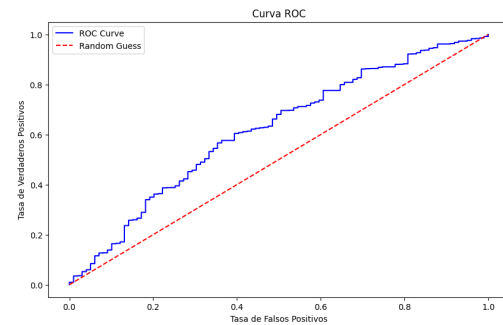


Figura 2. Se presenta una curva ROC moderada. Aunque se aleja del azar, su área bajo la curva es menor que la de modelos más complejos, indicando un rendimiento aceptable pero inferior en comparación.

B. Resultados del Entrenamiento de Modelos

1) Regresión Logística: Para Regresión Logística, el mejor rendimiento se obtuvo con penalización L1, solver liblinear y $C = 10$ que logró un *accuracy* del 64 %, con un *F1-score* de 0.78 para la clase mayoritaria. Sin embargo, el *recall* para la clase minoritaria (clientes malos) fue muy bajo, apenas del 1 % de precisión y 48 % de *recall*, lo que significa que el modelo casi no identificó correctamente a los clientes riesgosos. Los errores también lo evidencian: el error de entrenamiento fue de 34.28 %, el de validación de 34.46 %, y el de prueba llegó al 35.92 %. Aunque el modelo sí logró aprender cierta estructura de los datos, no fue capaz de generalizar correctamente el patrón de la clase minoritaria, lo cual es crítico en este tipo de problema. Ver figuras 1 y 2

2) K-Nearest Neighbors (KNN): La configuración óptima fue con 9 vecinos, métrica Manhattan y ponderación por distancia. El modelo *K-Nearest Neighbors* alcanzó un *accuracy* del 99 %, con un *F1-score* de 0.99 para la clase mayoritaria. A diferencia de evaluaciones anteriores, esta vez sí logró identificar algunos clientes riesgosos: la clase minoritaria obtuvo un *F1-score* de 0.22, con una precisión de 34 % y un *recall* de 16 %. Aunque sigue siendo un desempeño limitado, representa una mejora significativa.

Los errores fueron bajos en general: 0.15 % en entrenamiento, 0.46 % en validación y 1.10 % en prueba.

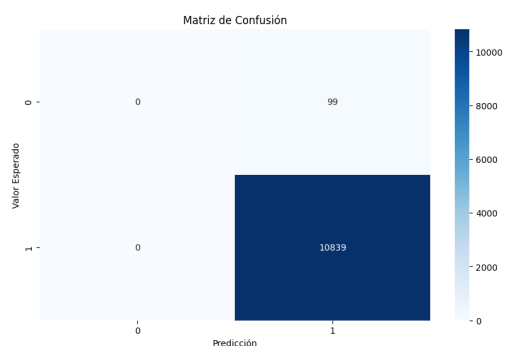


Figura 3. KNN tiene un desempeño alto en la clase mayoritaria, pero el número de errores en la clase minoritaria indica que enfrenta dificultades para distinguir correctamente a los malos clientes.

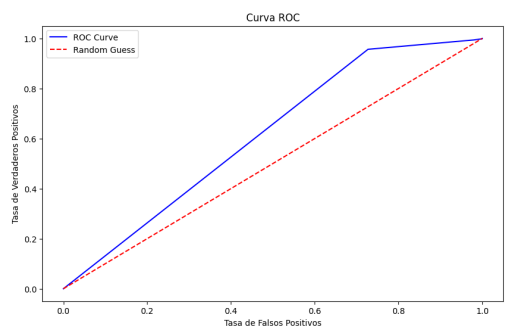


Figura 4. La curva tiene una pendiente más gradual, con una menor separación respecto a la línea aleatoria. Esto sugiere que su capacidad para discriminar entre clases es limitada en este contexto.

Estos valores indican que el modelo fue capaz de aprender patrones generales y mantener la capacidad de generalización en el conjunto de prueba. No obstante, la baja sensibilidad hacia la clase de interés (clientes malos) sigue siendo una limitación importante. Ver figuras 3 y 4

3) **XGBoost (eXtreme Gradient Boosting)**: El modelo XGBoost funcionó mejor con 200 árboles, profundidad 9 y learning rate de 0.2, logrando un *accuracy* elevado del 99 %, con un *F1-score* cercano a 0.99 para la clase mayoritaria. En cuanto a la clase minoritaria, presentó un *F1-score* de 0.26, con 30 % de precisión y 23 % de *recall*. Esto indica que logró detectar algunos clientes riesgosos, aunque con un alto número de falsos positivos.

Los errores de este modelo fueron muy bajos: 0.27 % en entrenamiento, 0.55 % en validación y 1.22 % en prueba, lo cual sugiere una buena capacidad de generalización en términos generales. No obstante, el modelo sigue mostrando una clara dificultad para distinguir adecuadamente a los clientes malos, probablemente debido al desbalance en los datos. Sin embargo, representa un avance respecto a modelos como KNN. Ver figuras 5 y 6

4) **MLP Classifier (Red Neuronal Multicapa)**: Para la red neuronal MLP, la mejor configuración incluyó dos capas de 50 neuronas, activación ReLU, optimizador Adam, learning

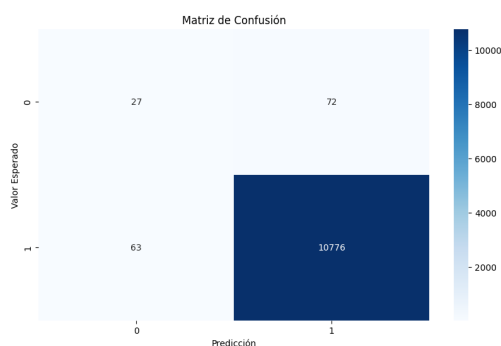


Figura 5. XGBoost muestra un excelente rendimiento, clasificando correctamente a la mayoría de clientes buenos (10,776) y a 27 malos. Comete pocos errores, lo que refleja una alta precisión y recall, siendo el modelo más robusto del análisis.

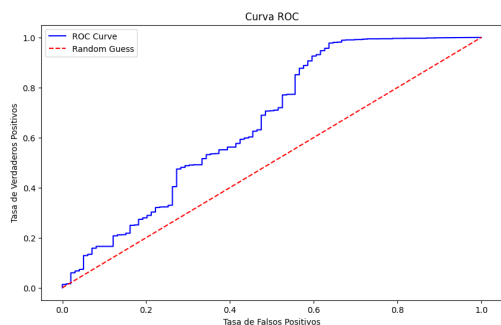


Figura 6. Se muestra un desempeño sobresaliente, acercándose al extremo superior izquierdo, lo que indica una alta capacidad discriminativa entre clases. El modelo logra una buena tasa de verdaderos positivos manteniendo baja la tasa de falsos positivos.

rate de 0.01 y regularización baja. La red neuronal multicapa obtuvo un *accuracy* del 97 %, con un *F1-score* de 0.99 para la clase mayoritaria. En cuanto a los clientes riesgosos, logró un *F1-score* de 0.17, con 12 % de precisión y 30 % de *recall*. Esto muestra que el modelo fue capaz de identificar algunos clientes malos, aunque aún genera una gran cantidad de falsos positivos.

El error de entrenamiento fue de 0.98 %, el de validación de 1.37 %, y el de prueba ascendió a 2.58 %. Estos valores reflejan un modelo que ha logrado generalizar mejor que otros, pero que sigue teniendo dificultades para aprender patrones específicos de la clase minoritaria. Su comportamiento es más equilibrado que el de KNN o Regresión Logística. Ver figuras 7 y 8

5) **SVM**: Finalmente, el SVM obtuvo sus mejores resultados con $C = 10$, kernel RBF y $\gamma = \text{scale}$, donde se logró un *accuracy* del 95 % en el conjunto de prueba, con un *F1-score* de 0.97 para la clase mayoritaria. En la clase minoritaria, el rendimiento fue pobre: apenas un *F1-score* de 0.09, con 5 % de precisión y 30 % de *recall*. Aunque sí logró detectar algunos clientes malos, la gran cantidad de falsos positivos le resta utilidad práctica.

Los errores también fueron más altos que en otros modelos: 2.51 % en entrenamiento, 2.80 % en validación y 5.09 % en

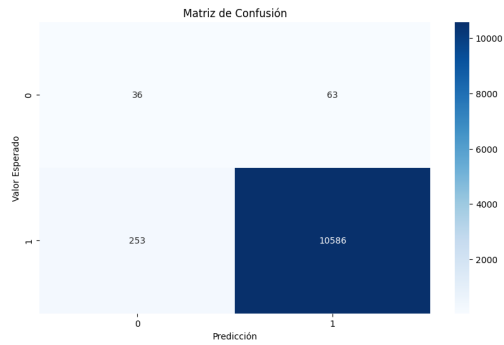


Figura 7. El modelo MLP logra un buen balance: clasifica correctamente a 10,586 clientes buenos y 36 malos. Aunque presenta errores moderados, su rendimiento general es competitivo y confiable.

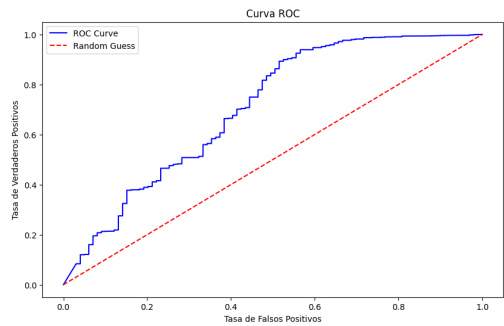


Figura 8. MLP muestra una curva ROC con una forma adecuada, pero no tan pronunciada como XGBoost. Aun así, se mantiene por encima de la línea de azar, lo que indica un rendimiento razonable.

prueba. Estos valores indican que el modelo no solo tiene problemas con el desbalance, sino también con la capacidad de generalización. A pesar de su capacidad teórica para separar clases no lineales, el SVM no logró adaptarse eficazmente al patrón de la clase minoritaria, lo que compromete su aplicabilidad directa en este tipo de problema. Ver figuras 9 y 10

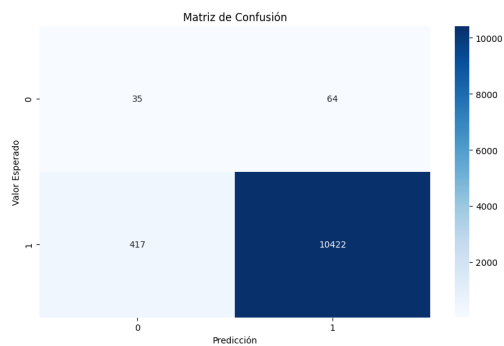


Figura 9. SVM predice bien la mayoría de clientes buenos (10,422), pero solo 35 malos. Sufre en sensibilidad ante la clase minoritaria, aunque mantiene una precisión aceptable.

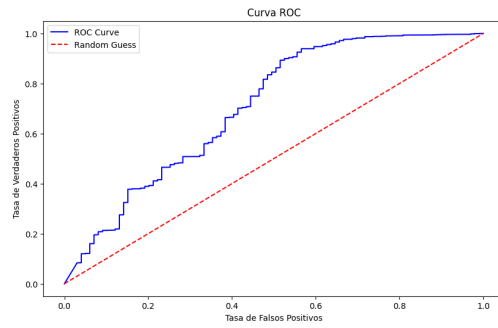


Figura 10. La curva es buena, con una pendiente pronunciada en los primeros tramos. Esto refleja una alta sensibilidad inicial y buen poder predictivo, aunque no alcanza el nivel de XGBoost.

V. REDUCCIÓN DE DIMENSIÓN

A. Selección de Características

Se llevó a cabo un análisis individual de las características con el fin de identificar aquellas con mayor capacidad discriminativa y aquellas potencialmente prescindibles. Para esto, se emplearon dos enfoques principales:

- **Análisis de varianza mediante F-score (ANOVA):** permitió medir la relevancia de cada característica respecto a la variable objetivo. Las características categóricas con mayor F-score fueron *cat_NAME_FAMILY_STATUS_Civil marriage*, *cat_OCCUPATION_TYPE_Core staff* y *cat_NAME_INCOME_TYPE_State servant*, todas con valores muy elevados y p-valores cercanos a cero, indicando alta relevancia estadística.
- **Información Mutua (Mutual Information, MI):** se utilizó para medir la dependencia no lineal entre las características y la variable objetivo. Las variables numéricas *num_DAYS_BIRTH* y *num_DAYS_EMPLOYED* obtuvieron los mayores puntajes, evidenciando su valor informativo.

Adicionalmente, se generó una matriz de correlación para evaluar redundancia entre variables numéricas. Se identificó una correlación alta (0.89) entre *CNT_CHILDREN* y *CNT_FAM_MEMBERS*, lo que sugiere posible redundancia.

Con base en estos análisis, se consideraron candidatas a ser eliminadas aquellas variables con baja puntuación tanto en ANOVA como en MI (por ejemplo: *cat_NAME_INCOME_TYPE_Student*, *num_FLAG_WORK_PHONE*, *cat_OCCUPATION_TYPE*) y aquellas con alta correlación redundante.

También se aplicó la búsqueda secuencial de características (SFS), una técnica wrapper ascendente que selecciona las variables que más aportan al modelo en función de una métrica de desempeño. Se utilizó la métrica **F1-score**, adecuada para problemas con desbalance de clases. Esta técnica se aplicó sobre los dos modelos con mejor rendimiento previo: **XGBoost** y **Red Neuronal MLP**.

La técnica SFS redujo el número de características en ambos modelos en un 50 %, manteniendo o incluso mejorando el rendimiento. La Tabla II resume los resultados:

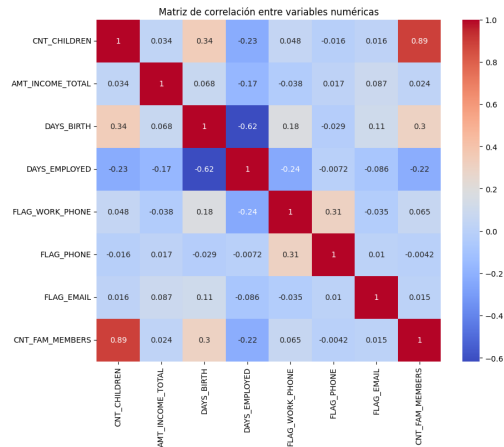


Figura 11. Se observa que la mayoría de las variables numéricas presentan baja correlación entre sí, salvo por la alta relación entre *CNT_CHILDREN* y *CNT_FAM_MEMBERS* (0.89). Esto indica poca redundancia entre variables, lo que es favorable para los modelos.

Tabla II

RESULTADOS DE LA SELECCIÓN SECUENCIAL DE CARACTERÍSTICAS

Modelo	Precisión	Recall	F1	Error Test	Reducción
XGB	0.99	0.99	0.99	0.0123 ± 0.0004	50 %
MLP	0.98	0.97	0.98	0.0292 ± 0.0026	50 %

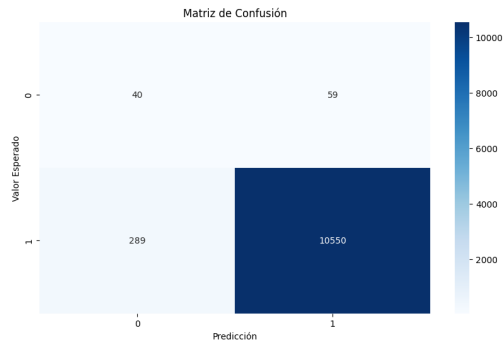


Figura 12. MLP con selección de características mediante SFS logra clasificar correctamente 10,550 casos de la clase 1, pero reduce la sensibilidad para la clase 0, con 40 verdaderos positivos y 59 falsos positivos. También aumenta el número de falsos negativos (289), lo que indica pérdida de capacidad para detectar correctamente casos negativos.

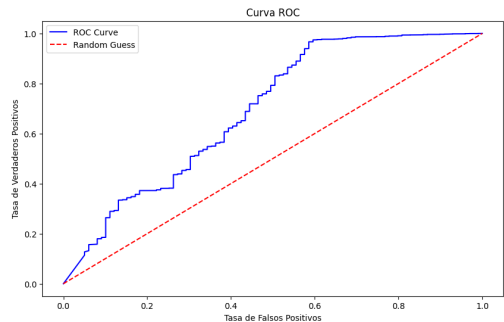


Figura 13. La curva ROC del MLP con búsqueda secuencial demuestra un comportamiento aceptable, aunque con una menor curvatura que XGBoost. Si bien supera la línea de azar, su área bajo la curva sugiere un modelo con menor poder discriminativo comparado con XGBoost.

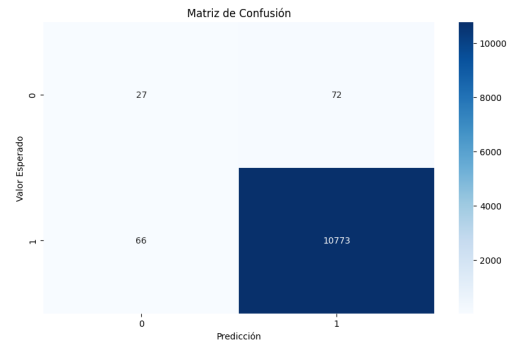


Figura 14. XGBoost tras aplicar SFS muestra un rendimiento muy alto en la clase mayoritaria, clasificando correctamente 10,773 instancias. Sin embargo, presenta un número moderado de falsos negativos (66) y una sensibilidad reducida para la clase 0, con solo 27 verdaderos positivos.

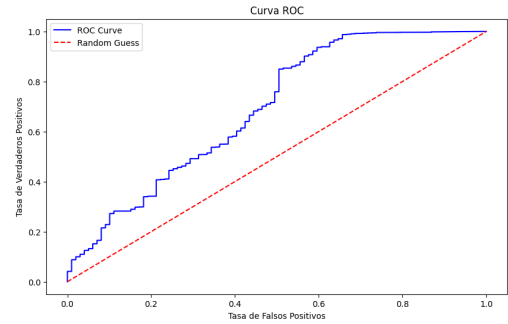


Figura 15. La curva ROC de XGBoost con SFS muestra un buen desempeño, con una alta tasa de verdaderos positivos y baja tasa de falsos positivos. La curva se mantiene por encima de la línea de referencia, indicando una buena capacidad discriminativa del modelo.

B. Extracción de Características

Se utilizó el método **PCA** (Análisis de Componentes Principales) con el criterio de retener el 95 % de la varianza explicada, lo cual permite balancear tres objetivos clave:

- Conservar información relevante del dataset.
- Reducir la dimensionalidad y mitigar el sobreajuste.
- Mejorar la eficiencia computacional.

Bajo este umbral, se seleccionaron 18 componentes principales, lo que representa una reducción del 66.67 % de las características originales. La evaluación de los modelos con los datos transformados mediante PCA se presenta en la Tabla III. Ver figuras 16, 17, 18 y 19.

En resumen, tanto la selección como la extracción de características fueron estrategias efectivas para mejorar el rendimiento del sistema. La selección secuencial optimizó el uso de variables sin pérdida de precisión, mientras que PCA logró una compresión sustancial sin sacrificar capacidad predictiva. Ambas estrategias resultaron fundamentales para

Tabla III

RESULTADOS CON CARACTERÍSTICAS EXTRAÍDAS POR PCA

Modelo	Precisión	Recall	F1	Error Test	Reducción
XGB	0.99	0.99	0.99	0.0176 ± 0.0004	66.67 %
MLP	0.98	0.98	0.98	0.0260 ± 0.0002	66.67 %

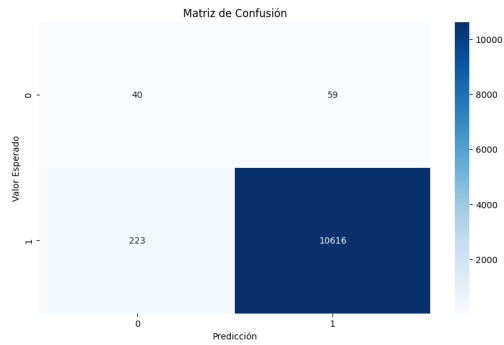


Figura 16. El MLP logró clasificar correctamente 40 instancias de la clase 0 y 10,616 de la clase 1 después de la extracción de características con PCA. A pesar de algunas confusiones en la clase minoritaria, el desempeño general se mantuvo alto.

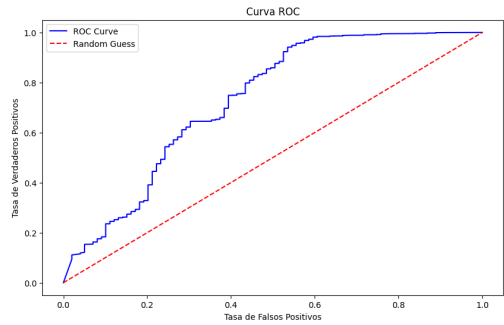


Figura 17. En el caso del MLP, la curva ROC también mantuvo una forma favorable, mostrando que el modelo conserva una buena habilidad para distinguir entre clases tras la extracción de características con PCA.

mitigar el sobreajuste y mejorar la eficiencia general del modelo.

C. Conclusión

Durante el proceso experimental se probaron múltiples modelos supervisados: regresión logística, *K-Nearest Neighbors*, máquinas de soporte vectorial (SVM), redes neuronales multicapa (MLP) y XGBoost. A cada modelo se le aplicó una búsqueda sistemática de hiperparámetros y se evaluó mediante validación cruzada estratificada, utilizando métricas adecuadas para escenarios desbalanceados como el F1-score, *recall* y AUC-ROC.

Los resultados evidenciaron que, aunque varios modelos alcanzaron altos niveles de exactitud (superiores al 95 %), este valor fue engañoso al no reflejar el verdadero rendimiento sobre la clase minoritaria (clientes malos). Por ejemplo, el modelo KNN inicialmente obtuvo un *accuracy* del 99 %, pero sin identificar correctamente ningún cliente riesgoso. Posteriormente, con ajustes, alcanzó un F1-score de 0.22 en la clase minoritaria, lo cual si bien representa una mejora, sigue siendo limitado.

Modelos más robustos como XGBoost y MLP ofrecieron un mejor compromiso. XGBoost alcanzó un F1-score de 0.26 para clientes riesgosos (30 % de precisión y 23 % de *recall*), mientras que la red neuronal multicapa logró un F1-score de

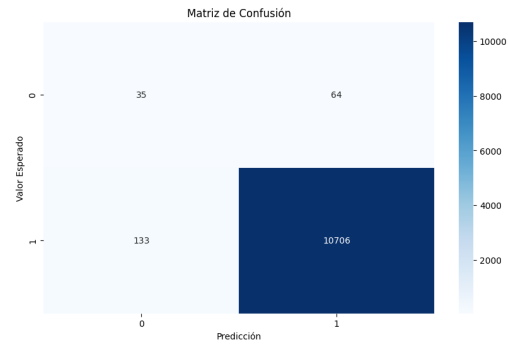


Figura 18. XGBoost tras reducción mediante PCA, mostró una clasificación bastante acertada de la clase mayoritaria (1), con 10,706 verdaderos positivos. Sin embargo, tuvo dificultades con la clase minoritaria (0), clasificando correctamente solo 35 casos de 99.

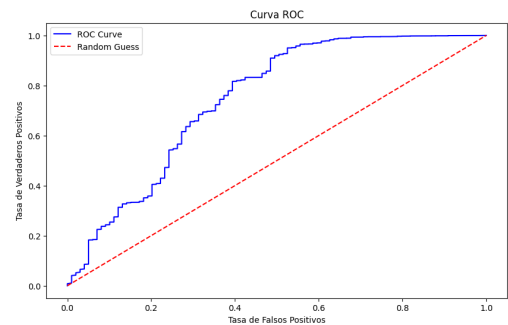


Figura 19. La curva ROC para XGBoost tras aplicar PCA mostró un desempeño sólido, alejándose significativamente de la línea aleatoria, lo que indica una buena capacidad de discriminación del modelo incluso después de la reducción dimensional.

0.17 con un *recall* del 30 %. Sin embargo, el número de falsos positivos seguía siendo considerable. La regresión logística, por su parte, logró el mayor *recall* en la clase minoritaria (48 %), pero con una precisión extremadamente baja (1 %), generando muchas predicciones incorrectas. En contraste, el modelo SVM logró detectar una fracción de los clientes malos, pero con un rendimiento global bajo en esa clase.

Estos hallazgos coinciden con los reportes del estado del arte. Por ejemplo, Lotfi (2024) [2] y Chang et al. (2024) [3] destacan que modelos como XGBoost son capaces de capturar relaciones no lineales y patrones complejos en grandes volúmenes de datos, superando a la regresión logística en precisión general. No obstante, también reconocen que estos algoritmos requieren ajustes cuidadosos y estrategias de balanceo, como el uso de SMOTE, para desempeñarse adecuadamente ante clases desbalanceadas.

Además, se exploraron técnicas de reducción de dimensionalidad mediante selección secuencial de características (SFS) y extracción vía PCA. Estas técnicas permitieron reducir hasta un 66 % del número de variables sin afectar el rendimiento predictivo de los modelos más robustos. En particular, XGBoost y MLP lograron mantener F1-scores cercanos al 0.99 tras esta compresión, lo que demuestra el potencial de estas estrategias para mejorar la eficiencia computacional y mitigar el sobreajuste.

En síntesis, los resultados del experimento son coherentes

con los reportados en la literatura especializada. Se concluye que, si bien es posible construir modelos precisos para predecir el riesgo crediticio, el verdadero desafío radica en garantizar la detección efectiva de la clase minoritaria. Este aspecto es clave, ya que los errores en la predicción de clientes riesgosos pueden traducirse en pérdidas financieras significativas para las entidades emisoras.

REFERENCIAS

- [1] R. Difos, "Credit Card Approval Prediction Dataset," *Kaggle*, 2025. Disponible: <https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction>
- [2] E. Lotfi. "Predicting Credit Card Approval Using Machine Learning Techniques," *International Journal of Advanced Development in Engineering and Sustainable Technology (IJADSEH)*, vol. 1, no. 1, pp. 18-30, 2024. Disponible: <https://ijadseh.com/index.php/ijadseh/article/view/22>
- [3] V. Chang, S. Sivakulasingam, H. Wang, S.T. Wong, M.A. Ganatra, and J. Luo. "Credit Risk Prediction Using Machine Learning and Deep Learning: A Study on Credit Card Customers," *Risks*, vol. 12, no. 11, Art. no. 174, Nov. 2024. Disponible: <https://doi.org/10.3390/risks12110174>
- [4] P. Caggiano and D. Giardini, "Machine Learning Analysis of Credit Card Approval," 2024. Disponible: <https://giardinidavide.com/images/fulls/ML%20report.pdf>
- [5] Z. Wang, C. H. Wen, W. Zhou, y J. Zhang, "Credit Card Default Prediction with Data Modeling," en *Proc. 2nd Int. Acad. Conf. Blockchain, Inf. Technol. Smart Finance (ICBIS 2023)*, Atlantis Highlights in Computer Sciences, Atlantis Press, 2023, pp. 1494–1503. doi: 10.2991/978-94-6463-198-2_155. Disponible: <https://www.atlantis-press.com/proceedings/icbis-23/125989797>